

Collapsing Strongly Connected Components into Risk Subgraphs with Structure-Aware Monte Carlo Graph Search

Anonymous ACL submission

Abstract

Long documents such as contracts, statutes, software package metadata, and corporate filings often contain risks that emerge only across mutually dependent records. LLM-guided Monte Carlo tree search (MCTS) can improve structured document reasoning, but standard tree rollouts become unstable on strongly connected components (SCCs), where the same logical record may be revisited indefinitely. We propose SA-MCGS, a structure-aware Monte Carlo graph search method that localizes SCCs, performs local LLM rollouts with relation-first evidence memory, and collapses each cyclic region into an inspectable risk subgraph. We evaluate a locked critical structural benchmark covering four authority-backed domains: Debian dependencies, SEC EX-21 subsidiary disclosures, German Civil Code cross-references, and CUAD contracts. Across 320 model-case pairs and four LLMs, SA-MCGS improves strict Root@3 from 48% to 81%, Risk-any from 72% to 98%, and Risk-all from 47% to 78%. The gains are larger under long-context pressure, while the returned subgraphs retain complete critical evidence and remove more than half of the original SCC nodes.

1 Introduction

Many high-stakes document analysis tasks are graph problems hidden inside text (Hendrycks et al., 2021; Koreeda and Manning, 2021; Boulet et al., 2021). A contract clause may define a deadline, another clause may suspend it, and a later amendment may restore only part of the obligation. A statute may create an exception whose scope is limited by a distant cross-reference (Boulet et al., 2021; Holzenberger et al., 2020). A software package may preserve an old interface while a dependent package requires the new one to be present first (Debian Policy Manual, 2025a,b). Each record can look plausible in isolation; the

risk appears only when the directed dependencies among records are followed.

Large language models (LLMs) can read long contexts, but nominal context length does not guarantee reliable retrieval, multi-hop tracing, or aggregation under long inputs (Liu et al., 2024; Hsieh et al., 2024; Bai et al., 2024). Legal NLP benchmarks expose persistent difficulty with distributed evidence (Hendrycks et al., 2021; Koreeda and Manning, 2021; Chalkidis et al., 2022; Guha et al., 2023). The failure mode we study is sharper: *cyclic* document regions, where evidence is revisitable and diffuse. A full-SCC one-shot prompt may find a suspicious node, but it must simultaneously rank all records, produce a subgraph, and preserve multiple evidence endpoints in one generation.

One might try to escape one-shot prompting by using AlphaGo-style search (Silver et al., 2016; Kocsis and Szepesvári, 2006; Browne et al., 2012). Nevertheless, ordinary MCTS is naturally a tree-unrolling procedure: in a loopy graph, the search state grows with walk prefixes as opposed to physical records. In an SCC, even a small set of records can therefore induce many repeated path copies. Related MCTS analyses describe this as a transposition, graph-history, and loop-induced uncertainty problem (Childs et al., 2008; Kishimoto and Müller, 2004; Saffidine et al., 2012; Moerland et al., 2020). Prior Monte Carlo graph search reduces duplicate transpositions for AlphaZero-style search by sharing states in a DAG-like search graph (Czech et al., 2021). Other graph-to-DAG strategies, such as removing or orienting feedback edges to obtain an acyclic approximation (Eades et al., 1993; Berger and Shor, 1990), avoid cycles by discarding part of the cyclic structure. Neither line provides a mechanism for turning a strongly connected document region itself into a stable and reliable evaluation unit. Our setting instead requires localizing the SCC and collapsing it into an

evidence-preserving risk subgraph, showing more detail in risk source and evidence.

We propose **SA-MCGS**, a structure-aware Monte Carlo graph search method for extracting risk subgraphs from cyclic document regions. SA-MCGS goal is finding SCC bottlenecks at first, then performs local window rollouts inside each SCC. Instead of treating risk as an intrinsic property of a single node, it stores relation-first evidence: a node becomes risky when earlier evidence makes its constraint, exception, or handoff incompatible with the graph context. The final output is a dynamic core risk subgraph that can be inspected or passed upward as a collapsed evaluation point.

Our contributions are:

- We formulate *structural risk subgraph extraction*: given an SCC in a document graph, output a compact subgraph that retains the repair-relevant evidence endpoints, in place of single anomalous node.
- We introduce SA-MCGS, a structure-aware Monte Carlo graph search method that localizes SCCs, accumulates relation-level evidence through local rollouts, and distills persistent signals into a compact dynamic risk core. Its rollout policy uses UCB selection augmented with progressive relation priors, critical-pair revisiting, and evidence-gated core pruning, so additional search budget is routed toward relations that repeatedly look repair-relevant rather than toward arbitrary path copies.
- We build a frozen critical-stress benchmark from four authority-backed sources. Debian provides package-dependency graphs; SEC EX-21 provides subsidiary-disclosure graphs; BGB provides statutory cross-reference graphs; CUAD provides contract-clause graphs. The benchmark contains 320 model-case pairs evaluated with GPT-4o, DeepSeek-V3, Qwen2.5-72B, and Gemini 2.5 Pro. Across these settings, SA-MCGS improves strict Root@3, Risk-any, and Risk-all over a full-SCC direct-subgraph baseline. It also makes the retention cost visible through an explicit compression metric.

2 Task Formulation

Record graph. Let $G = (V, E)$ be a directed graph whose nodes are document records and

whose edges denote typed dependencies such as reference, constraint, modification, trigger, or control. A record may be a clause, statute section, package metadata entry, subsidiary disclosure, or other domain unit. We use Tarjan’s algorithm (Tarjan, 1972) to decompose G into strongly connected components. A component $S \subseteq V$ is a reasoning bottleneck when every node in S can reach every other node through directed dependencies.

Structural risk. We study risks that arise from relations, not merely from suspicious wording. In the benchmark, an injected critical case modifies existing records only; no new nodes are added. The core evidence contains a root record, a witness record, and sometimes affected records. The root and witness can each be locally plausible, but jointly create an incompatible obligation, temporal condition, ownership statement, or dependency handoff.

Output. Given a strongly connected component S , the system returns a risk subgraph $H \subseteq S$ and, when applicable, a ranking over risky records. We evaluate:

- **Root@3:** whether the injected root appears in the top three ranked records.
- **Risk-any:** whether H contains at least one repair-relevant risk endpoint.
- **Risk-all:** whether H retains all core risk endpoints. This is strict but important for critical risks because remediation often requires seeing both sides of the contradiction.
- **Compression:** $1 - |H|/|S|$, reported higher-is-smaller. Compression is not optimized alone; a tiny subgraph that drops the evidence endpoints is not useful.

We report effective compression and effective OC as diagnostics in Appendix A; they are not primary success metrics.

3 Method

3.1 Why ordinary MCTS is not enough

Standard tree search treats a trajectory as a growing path, following the usual MCTS loop of selection, expansion, simulation, and backpropagation (Browne et al., 2012). In a directed acyclic graph this is acceptable: rollouts eventually terminate and backpropagation updates a finite set of

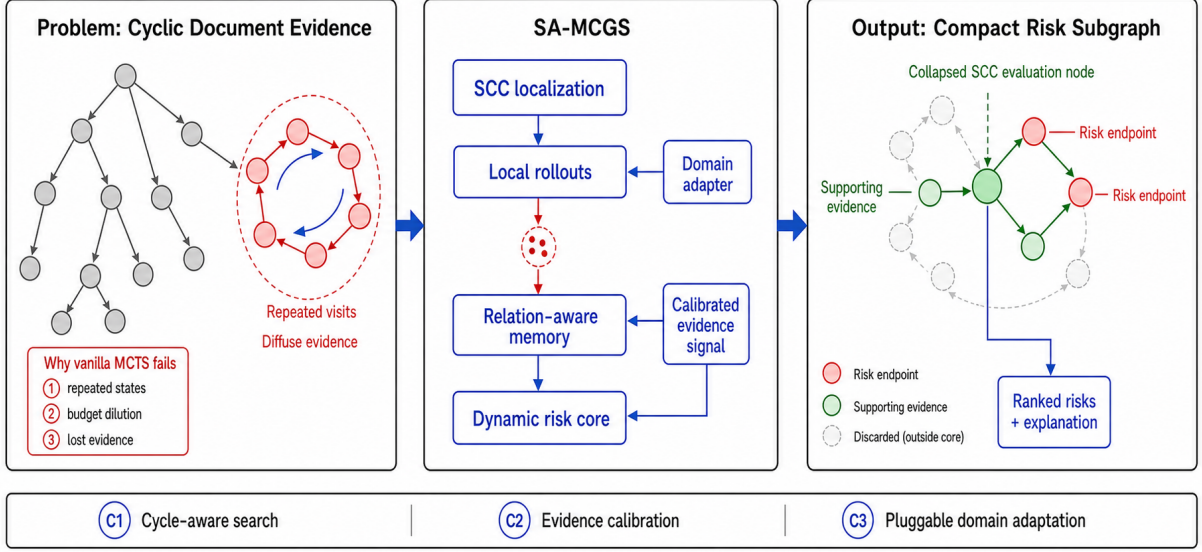


Figure 1: SA-MCGS pipeline: SCC localization, local rollouts, relation-first evidence memory, and dynamic-core collapse.

states. In an SCC, the same physical record can be revisited through many path copies (Childs et al., 2008). If a rollout path is $p = (v_1, \dots, v_\ell)$, ordinary TreeMCTS indexes states by path prefix. A physical node v can therefore appear as many distinct tree states,

$$\mathcal{P}_T(v) = \{p_{\leq t} : p_t = v, p \in \Pi_T\}, \quad (1)$$

where Π_T is the set of paths generated before budget T . The effective number of states can therefore grow with path copies instead of $|S|$. This causes search-tree expansion, rollout truncation, and value dilution.

Figure 2 illustrates the failure mode. The issue is not merely that a cycle exists; it is that tree expansion treats revisits as new path states. A long cyclic document region can therefore consume rollout budget while repeatedly rediscovering the same records, making it harder to preserve the endpoints that jointly create a structural risk.

We verify this failure mode with a controlled motivating ablation. We take matched document-graph components with the same local risk task and compare three search variants under the same rollout budget: ordinary TreeMCTS on an acyclic dependency graph, ordinary TreeMCTS on an SCC, and SA-MCGS on the same SCC setting. The ablation is intentionally small and diagnostic; its role is not to establish the final benchmark result, but to test whether SCC topology breaks the assumptions that make tree search attractive in DAG-like document graphs.

Method	Topology	Expansion	Trunc.	Q-spread	Hit@3
TreeMCTS	DAG	$2.3\times$	0%	0.077	7%
TreeMCTS	SCC	$6.4\times$	100%	0.044	30%
SA-MCGS	SCC	$1.0\times$	—	0.371	90%

Table 1: Motivating ablation: ordinary TreeMCTS expands path copies and dilutes value estimates inside strongly connected components, whereas SA-MCGS treats each SCC as a localized reasoning unit.

Table 1 tests the predicted failure directly: inside an SCC, TreeMCTS expands many more path-copy states, truncates every rollout, and produces flatter value estimates. SA-MCGS avoids path copies by indexing evidence by physical nodes and relations, giving $1.0\times$ expansion and a much larger Q-spread. Detailed definitions of Expansion, Truncation, Q-spread, and Hit@3 are given in Appendix A.

3.2 Objective

SA-MCGS avoids path-copy states by treating each SCC S as a localized graph object. The desired output is a subgraph $H \subseteq S$ that keeps enough evidence for remediation while remaining smaller than the original component. Let $R^* \subseteq S$ denote the latent repair-relevant endpoints in evaluation, e.g., root and witness records. At inference R^* is unknown, so the algorithm estimates node and relation utility from rollout evidence. The empirical objective is:

$$\max_{H \subseteq S} \hat{R}(H) - \lambda \frac{|H|}{|S|}, \quad (2)$$

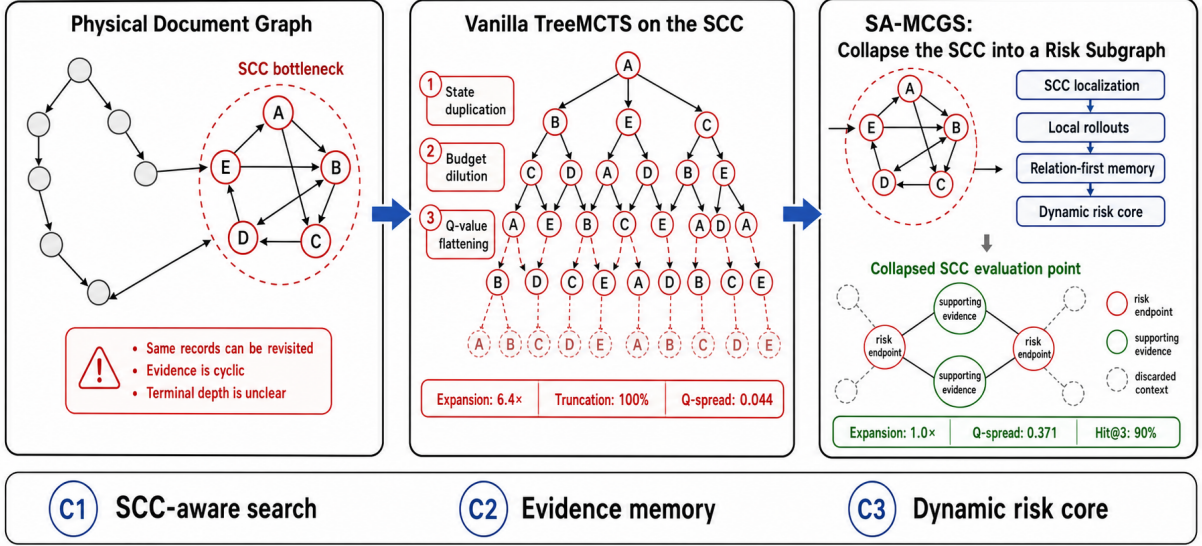


Figure 2: Why vanilla TreeMCTS is a poor primitive for SCCs: cyclic records create path-copy states, dilute rollout budget, and flatten value estimates.

where $\hat{R}(H)$ is estimated evidence retention and λ controls compression pressure. The first term corresponds to Risk-any and Risk-all; the second corresponds to compression. This formulation makes clear why compression alone is insufficient: a small H is useful only if it retains repair-relevant evidence.

3.3 SCC-aware local rollouts

We first decompose the record graph using Tarjan’s linear-time algorithm (Tarjan, 1972). DAG regions are handled by the ordinary upstream search, whereas each non-trivial SCC is processed by SA-MCGS. At rollout t , the algorithm selects a seed $a_t \in S$ and constructs a local window

$$W_t = \{v \in S : \text{dist}_S(a_t, v) \leq r\} \cup B_t, \quad (3)$$

where r is a small graph radius and B_t contains selected bridge or memory nodes. The LLM evaluator observes node text and visible directed edges restricted to W_t , excluding the remainder of the SCC. It returns local node scores $y_t(v) \in [0, 1]$, relation evidence $z_t(u, v) \in [0, 1]$, a local risk subgraph h_t , and optional OC evidence.

Seed selection and frontier expansion follow the UCB intuition used in MCTS (Kocsis and Szepesvári, 2006), but statistics are keyed by physical graph objects rather than path copies. Let x denote either a seed node v or a frontier edge $e = (u, v)$. SA-MCGS scores both with the same

exploration template:

$$U_t(x) = \bar{r}_t(x) + c_t \sqrt{\frac{\log(1 + N_t)}{n_t(x) + \ell_t(x) + \epsilon}} + \frac{\lambda \pi_t(x)}{1 + \delta \sqrt{n_t(x)}} + \eta_t(x), \quad (4)$$

where $\bar{r}_t(x)$ is the current empirical risk or conflict estimate, $n_t(x)$ is the number of times x has been observed, $\ell_t(x)$ is virtual loss for concurrent workers, $\pi_t(x)$ is a relation-first prior, and $\eta_t(x)$ is exploration noise. The prior is computed from rollout evidence rather than from domain-specific labels:

$$\pi_t(x) = \frac{[w^\top \phi_t(x)]_+}{\max_{x'} [w^\top \phi_t(x')]_+ + \epsilon}. \quad (5)$$

For nodes, $\phi_t(x)$ contains repair-entry counts, local risk-subgraph appearances, conflict-pair endpoint support, AMAF/RAVE-style evidence, path-fragment support, and OC endpoint indicators. For edges, it contains semantic incompatibility, repeated pair support, explicit conflict reports, reverse-edge support, and accumulated conflict evidence. The weights w are fixed before the locked evaluation and reported in Appendix F. The decay term in Eq. 4 prevents an early prior from permanently dominating once the node or edge has been sampled often enough. This implements progressive relation bias, analogous in spirit to progressive bias and RAVE-style reuse (Gelly and Silver, 2007; Chaslot et al., 2008; Browne et al.,

2012), while keeping the update key tied to physical nodes and edges rather than path prefixes (Czech et al., 2021).

3.4 Relation-first evidence memory

Structural risk is relational: a node can be harmless until previous evidence makes it incompatible with the graph context. This follows the relational-inductive-bias view that graph-structured reasoning should represent entities together with the relations that define their roles (Battaglia et al., 2018). We therefore maintain memory

$$M_t = \{n_t(v), \bar{q}_t(v), m_t(u, v), o_t(v)\}_{v, u \in S}, \quad (6)$$

where $m_t(u, v)$ stores support that the directed relation (u, v) makes either endpoint risky, and $o_t(v)$ stores whether v has appeared in effective OC evidence. Node evidence is updated by running averages and relation evidence by smoothed pair support; Appendix E gives the exact update equations. Let $r_t^-(v)$ and $r_t^+(v)$ be the strongest incoming and outgoing relation-memory scores around v . The priority score used for core construction combines node evidence, relation evidence, OC support, and a persistent critical-pair ledger:

$$s_t(v) = \bar{q}_t(v) + \alpha r_t^-(v) + \alpha r_t^+(v) + \gamma o_t(v) + \mu p_t(v). \quad (7)$$

Thus a node can become important because earlier or later evidence repeatedly implicates it, even if its isolated text is not obviously defective. Relation evidence is aggregated by maxima over incoming and outgoing edges to limit degree bias from highly connected nodes. The OC and pair-ledger terms are bounded bonuses: they route search toward severe recurring hypotheses, but do not let a single local judgment dominate persistent node or relation evidence. This is the persistence mechanism that prevents a single noisy local judgment from deciding the final core, while still letting strong recurring relations guide pruning.

3.5 Online conformal-style evidence signal

Single LLM judgments are noisy, especially on local windows. We therefore add a calibration-inspired discovery signal. At each rollout, SA-MCGS compares the strongest local relation-conflict score against an online quantile of previous conflict scores; Appendix E gives the exact rule. We call a threshold crossing an online calibrated conflict signal, abbreviated OC: it indicates

that the current local window contains an unusually strong ordered-cycle or contradiction cue relative to earlier rollouts. When this happens, endpoints of the implicated relation receive an OC bonus in Eq. 7. This follows the spirit of conformal calibration (Vovk et al., 2005; Gibbs and Candès, 2021), but we do not claim formal coverage because the LLM evaluator and rollout policy are adaptive. We use OC as evidence routing and convergence support, while final evaluation is still based on the returned subgraph H .

3.6 Dynamic core compression

The final risk subgraph is maintained as a dynamic core with replacement, avoiding monotonic expansion. The compression step is a budgeted subset-selection problem; greedy selection is a standard approximation strategy for monotone submodular objectives under cardinality constraints (Nemhauser et al., 1978). After each rollout, SA-MCGS forms a candidate pool C_t from the local risk subgraph, the previous core, OC endpoints, and top-scoring relation endpoints with local neighbors. The core is selected by maximizing a bounded utility:

$$H_t = \arg \max_{\substack{H \subseteq C_t \\ |H| \leq B(S)}} \left[\sum_{v \in H} s_t(v) + \eta \sum_{(u, v) \in E(H)} m_t(u, v) - \lambda |H| \right]. \quad (8)$$

We implement this objective with greedy replacement: high-confidence endpoints may evict weaker nodes, whereas repeatedly supported relation endpoints are retained as locked evidence. This persistence mechanism is essential for critical cases. When a rollout observes both endpoints of a severe contradiction, subsequent rollouts should revisit and stabilize that relation, instead of allowing unrelated high-scoring distractors to occupy the subgraph.

3.7 Computational feasibility

SA-MCGS is an anytime graph-search procedure in the sense of Zilberstein (1996). Tarjan preprocessing is $O(|V| + |E|)$. For a component S , the method performs T local LLM evaluations over windows of at most w records, with prompt cost $O(T \cdot \text{tokens}(w))$. Its evidence memory is

Algorithm 1 SA-MCGS for one strongly connected component

Require: Component $S = (V_S, E_S)$, budget T , shared rubric and domain serializer

```

1: Initialize memory  $M_0$  and core  $H_0 = \emptyset$ 
2: for  $t = 1$  to  $T$  do
3:   Select seed  $a_t = \arg \max_{v \in V_S} U_t(v)$  using Eq. 4
4:   Build local window  $W_t$  around  $a_t$  plus memory bridges
5:   Query LLM with the shared risk rubric on  $W_t$ 
6:   Receive  $y_t(v)$ ,  $z_t(u, v)$ , local subgraph  $h_t$ , and OC evidence
7:   Update node and relation memory  $M_t$ 
8:   Recompute dynamic core  $H_t$  using Eq. 8
9: end for
10: return risk subgraph  $H_T$  and node ranking by  $s_T(v)$ 

```

keyed by physical nodes and observed relations, not path-copy states, and stores $O(|S| + |E_S|)$ statistics. The dynamic core cap $B(S)$ bounds the retained subgraph, so additional budget refines the same physical memory rather than expanding a cyclic tree indefinitely. This yields an anytime collapsed evaluation point for higher-level graph search; budget-prefix behavior is analyzed in Figure 4 and Appendix A.

4 Experimental Setup

4.1 Domains

We evaluate on four domains chosen for structural diversity and source authority, not for result cherry-picking. Table 2 summarizes their roles.

4.2 Domain formatting and shared risk semantics

The four sources differ in surface form: a Debian record exposes package fields, a BGB record is a statutory section, an SEC EX-21 record is a subsidiary disclosure, and a CUAD record is a contract clause. We therefore use lightweight domain formatters

$$\mathcal{A}_d = (\text{serialize}_d, \text{edges}_d, \text{normalize}_d) \quad (9)$$

to serialize records, expose typed directed edges, and normalize model outputs. These formatters do not define the target risk type and do not reveal the injected endpoints. Both Naive and SA-MCGS use the same domain-neutral risk rubric: identify structural inconsistency, mutual incompatibility, underspecification, or high-impact risk from visible node text and visible directed edges. Thus the comparison is not between different notions of risk. The difference is computational: full-SCC

one-shot generation versus SCC-aware rollout selection, relation-first memory, critical-pair revisiting, and dynamic-core compression.

4.3 Critical structural injection

The main experiment uses a locked critical-stress configuration over the same SCC blocks and models for both methods; the exact run profile is reported in Appendix J. Each case edits only existing records in a real SCC. A root record and a distant witness or affected record jointly create a structural conflict, while each record remains locally plausible. Both methods receive the same generic risk semantics and the same visible graph; the exact templates and domain examples are in Appendix I.

4.4 Baselines and models

We compare against a full-context direct-subgraph baseline: Naive sees the full SCC in one prompt and is asked to output both a whole-SCC risk ranking and a risk subgraph. SA-MCGS sees smaller local windows over 60 rollouts and aggregates evidence into a dynamic core. Thus Naive has more context per call but carries a heavier one-shot output burden, while SA-MCGS trades this for iterative search and evidence retention.

We evaluate four models: GPT-4o, DeepSeek-V3, Qwen2.5-72B-Instruct, and Gemini 2.5 Pro. The main data contain 320 model-case pairs and 640 method-level records.

5 Main Results

Figure 3 shows the core result. SA-MCGS improves Root@3 by 32 points, Risk-any by 26 points, and Risk-all by 31 points. The strongest claim is not that one-shot LLMs never find risk; the Naive baseline often finds at least one suspicious record. The difference is that SA-MCGS more reliably preserves the repair-relevant evidence endpoints in the final subgraph.

The compression axis should be read as a trade-off, not a loss. Naive returns smaller subgraphs on average, but it often drops one side of a critical contradiction or produces an output that is too large, incomplete, or otherwise unavailable for scoring. SA-MCGS deliberately keeps a larger dynamic core when evidence persists across rollouts.

Strict rates keep unavailable structured outputs in the denominator; Appendix 9 reports the output-availability audit.

Domain	Source authority	Graph role	Paper role
Debian	Package dependency metadata	Software migration / dependency constraints	Non-legal technical generalization
SEC EX-21	Public subsidiary exhibit structure	Entity-control and consolidation paths	Financial/legal entity generalization
BGB	German Civil Code references	Statutory cross-reference cycles	Clean legal-rule stress test
CUAD	Expert contract review dataset	Long contract clause graph with ambiguity	Noisy contract-domain stress test

Table 2: Domain coverage. The benchmark spans technical dependencies, public regulatory disclosures, official statutory references, and expert-labeled commercial contracts.

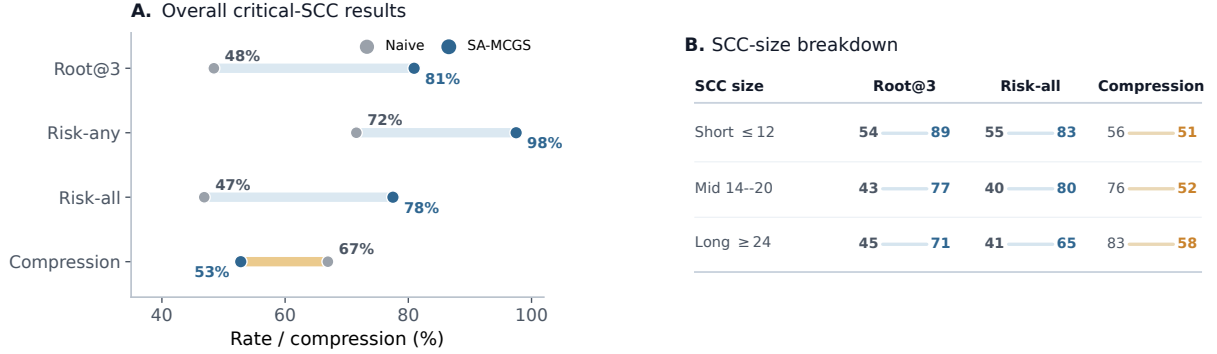


Figure 3: Main critical-SCC results on 320 model-case pairs per method. Panel A reports strict headline metrics; Panel B gives a compact SCC-size breakdown of endpoint retention and compression. Compression is higher-is-smaller and therefore shows the cost of retaining more evidence.

5.1 Breakdown by model and domain

The gains are not carried by a single model. GPT-4o and Qwen2.5-72B show large Risk-all gains, DeepSeek-V3 is already strong under Naive but still improves, and Gemini 2.5 Pro is competitive for both methods. Domain-wise, CUAD is the hardest setting because long contract text creates high output burden for full-SCC prompting; Debian and SEC are cleaner but still show endpoint-retention gains.

6 Analysis

6.1 Effect of SCC size

Figure 3B separates the results by SCC size. In short components (≤ 12 nodes), SA-MCGS improves Risk-all from 55% to 83%. In mid-sized components (14–20 nodes), the improvement is 40% to 80%. In long components (≥ 24 nodes), Risk-all remains difficult, but SA-MCGS still improves 41% to 65%. This is consistent with the task: longer cycles contain more plausible distractors and more native ambiguity, especially in CUAD.

6.2 Rollout convergence

Figure 4 shows that additional rollouts do not merely spend more tokens; they accumulate useful evidence. At budget 5, Risk-all is 22%; at budget 30, it rises to 41%; at budget 60, it reaches 57% under prefix reconstruction. The final run-level Risk-all in Figure 3 is higher because the dynamic core uses the full retained evidence rather than a simple prefix truncation. The important trend is that risk coverage and Risk-all continue to rise well past the first few rollouts, supporting the search interpretation.

6.3 Robustness checks

Appendix J reports two auxiliary checks. First, a more compressed profile reduces the average retained core from 7.69 to 5.91 nodes but lowers Risk-all from 78% to 71%, so the main run keeps the profile that better preserves critical endpoints. Second, strict rates do not hide unavailable Naive outputs: on 261 matched usable pairs, SA-MCGS remains higher in Root@3 (83% vs. 59%), Risk-any (98% vs. 88%), and Risk-all (77% vs. 57%). A CUAD lite-output control likewise reduces formatting burden but still reaches only 12% Risk-all on long-contract SCCs, indicating that full-SCC prompting is both an output and reasoning burden.

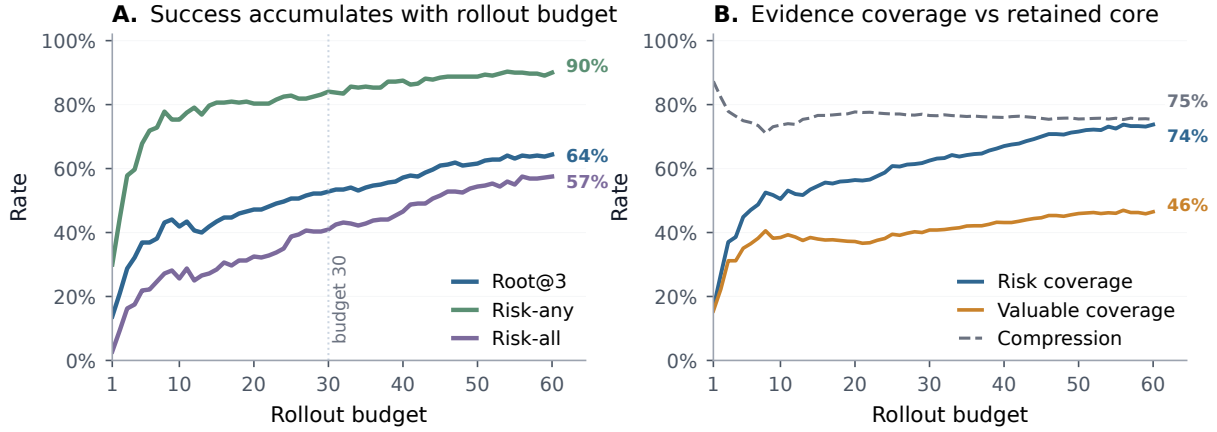


Figure 4: Budget-prefix convergence: localization and evidence coverage improve with rollout budget while the retained core stays compact.

Appendix C gives a representative long-SCC case study.

7 Related Work

LLM reasoning and long context. Chain-of-thought prompting improves sequential reasoning (Wei et al., 2022), but cyclic document risk requires revisiting and reconciling records rather than extending one chain. Long-context benchmarks show persistent failures in retrieval, multi-hop tracing, and aggregation (Liu et al., 2024; Hsieh et al., 2024; Bai et al., 2024), while graph-reasoning studies show sensitivity to graph encoding and task formulation (Wang et al., 2023; Fatemi et al., 2024). Legal NLP benchmarks motivate document-level reasoning (Hendrycks et al., 2021; Koreeda and Manning, 2021; Chalkidis et al., 2022; Guha et al., 2023); our task stresses cyclic structural evidence retention.

Search-augmented LLMs. Tree of Thoughts (Yao et al., 2024), Graph of Thoughts (Besta et al., 2024), and planning-style LLM search (Hao et al., 2023) show that explicit search can improve reasoning. SA-MCGS operates on externally constructed document graphs and treats SCCs as bottlenecks to collapse into risk subgraphs.

Monte Carlo search on graphs. MCTS and AlphaGo-style search are powerful under stable state transitions (Silver et al., 2016; Kocsis and Szepesvári, 2006; Browne et al., 2012). Monte Carlo graph search reuses transpositions (Childs et al., 2008; Czech et al., 2021), and work on MCTS uncertainty highlights loop difficulty (Møerland et al., 2020). Our method targets the re-

maining gap: information-preserving SCC handling for cyclic document regions.

Graph legal analysis and calibration. Network analysis has been used for statutory cross-references (Boulet et al., 2021), contract graph extraction is active (Dechtiar et al., 2025), and conformal ideas inspire calibrated evidence signals (Vovk et al., 2005; Gibbs and Candès, 2021). SA-MCGS is complementary: it assumes a record graph and focuses on risk search inside cyclic components.

8 Conclusion

We introduced SA-MCGS, a structure-aware Monte Carlo graph search method for cyclic document risk. The central idea is to stop treating a long SCC as either a flat prompt or an ordinary tree-search path. Instead, SA-MCGS localizes the SCC, searches it through local rollouts, stores relation-first evidence, and collapses the component into a dynamic risk core. Across four domains and four LLMs, this yields substantially stronger root localization and endpoint retention than a full-SCC direct-subgraph baseline, while making the cost in compression explicit.

Limitations

SA-MCGS is more expensive than one-shot prompting because it performs many local LLM calls. The method is intended for high-risk cyclic components, not every record in a document. Our benchmark uses injected critical risks so that ground truth is controlled; naturally occurring risks may be messier and require human audit. Graph construction quality also matters: missing

or spurious edges can change the SCCs that the method receives. Finally, Risk-all remains difficult in noisy long-contract settings, which is why we report it explicitly rather than replacing it with the easier Risk-any metric.

References

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics.

Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, and 1 others. 2018. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*.

Bonnie Berger and Peter W. Shor. 1990. [Approximation algorithms for the maximum acyclic subgraph problem](#). In *Proceedings of the First Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 236–243. Society for Industrial and Applied Mathematics.

Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoeftler. 2024. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38.

Romain Boulet, Pierre Music, Corinna Coupette, Dirk Hartung, Michael Bommarito, and Daniel Martin Katz. 2021. [Measuring law over time: A network analytical framework with an application to statutes and regulations in the United States and Germany](#). *Frontiers in Physics*, 9:658463.

Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Bohnlshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. 2012. [A survey of Monte Carlo tree search methods](#). *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43.

Ilias Chalkidis, Abhik Jana, Dirk Hartung, Michael Bommarito, Ion Androutsopoulos, Daniel Martin Katz, and Nikolaos Aletras. 2022. LexGLUE: A benchmark dataset for legal language understanding in english. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4310–4330. Association for Computational Linguistics.

Guillaume M. J.-B. Chaslot, Mark H. M. Winands, H. Jaap van den Herik, Jos W. H. M. Uiterwijk, and Bruno Bouzy. 2008. [Progressive strategies for Monte-Carlo tree search](#). *New Mathematics and Natural Computation*, 4(3):343–357.

Benjamin E. Childs, James H. Brodeur, and Levente Kocsis. 2008. Transpositions and move groups in Monte Carlo tree search. In *2008 IEEE Symposium On Computational Intelligence and Games*, pages 389–395. IEEE.

Johannes Czech, Patrick Korus, and Kristian Kersting. 2021. Monte-carlo graph search for AlphaZero. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 31, pages 103–111.

Debian Policy Manual. 2025a. [Declaring relationships between packages](#). Version 4.7.2.0.

Debian Policy Manual. 2025b. [Shared libraries](#). Version 4.7.2.0.

Moriya Dechter, Daniel Martin Katz, Mari Sundaresan, Sylvain Jaume, and Hongming Wang. 2025. GRAPH-GRPO-LEX: Contract graph modeling and reinforcement learning with group relative policy optimization. *arXiv preprint arXiv:2511.06618*.

Peter Eades, Xuemin Lin, and W. F. Smyth. 1993. [A fast and effective heuristic for the feedback arc set problem](#). *Information Processing Letters*, 47(6):319–323.

Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2024. Talk like a graph: Encoding graphs for large language models. In *The Twelfth International Conference on Learning Representations*.

Sylvain Gelly and David Silver. 2007. [Combining online and offline knowledge in UCT](#). In *Proceedings of the 24th International Conference on Machine Learning*, pages 273–280. ACM.

Isaac Gibbs and Emmanuel Candès. 2021. Adaptive conformal inference under distribution shift. In *Advances in Neural Information Processing Systems*, volume 34, pages 1660–1672.

Neel Guha, Julian Nyarko, Daniel E. Ho, and 1 others. 2023. LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models. *arXiv preprint arXiv:2308.11462*.

Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. 2023. Reasoning with language model is planning with world model. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 8154–8173. Association for Computational Linguistics.

Dan Hendrycks, Collin Burns, Anya Chen, and Spencer Ball. 2021. CUAD: An expert-annotated NLP dataset for legal contract review. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*.

suggestions, SA-MCGS cores, Naive subgraphs, or terms such as root, witness, affected node, SCC, and risk subgraph. Hidden fields are used only after annotation to compare expert labels with injected endpoints and system outputs.

All three annotators hold doctoral degrees in relevant fields and have professional experience with legal, regulatory, or contractual analysis.

Case-level task. For each case, the expert reads a group of related paragraphs and answers whether the group contains a structural conflict. A structural conflict means that two or more paragraphs cannot all be satisfied together, leave a high-impact condition unresolved, or make a repair decision depend on inconsistent assumptions.

Paragraph-level task. For each paragraph, the expert chooses one of four labels:

- **Clear issue:** the paragraph is itself problematic, or clearly conflicts with another visible paragraph.
- **Possibly relevant:** the paragraph may be needed to understand or repair the conflict, but is not clearly wrong by itself.
- **No issue observed:** the paragraph appears to be ordinary context.
- **Uncertain:** the expert cannot decide without more context or specialist knowledge.

Repair-material task. After labeling paragraphs, the expert selects which paragraphs should be included in the final repair packet. This task is stricter than merely identifying a suspicious sentence: the packet should include enough material for a reviewer to understand and fix the conflict.

Annotation protocol. Each sample is annotated independently by three experts. Every sample therefore receives three case-level decisions, three sets of paragraph-level labels, and three repair-packet selections. Disagreements are preserved for analysis rather than forced into an immediate majority vote.

Audit results. Table 3 summarizes the completed annotations. All 12 audited cases receive a majority structural-conflict label. Experts also recognize the injected evidence endpoints: 23 of 24 root/witness endpoint paragraphs receive a majority *clear issue* label, and all 24 receive a major-

ity *clear issue* or *possibly relevant* label. Agreement is strongest for the clear-risk endpoint decision, with Fleiss’ $\kappa = 0.606$ and pairwise Cohen’s κ ranging from 0.452 to 0.759. These results support that the injected critical risks are semantically visible to qualified readers when the relevant paragraphs are inspected.

The repair-packet task is intentionally stricter and more variable. One annotator selected no individual paragraph for the final packet while still marking every case as conflicting and most cases as sufficiently or partially supported. We therefore treat repair-packet selection as a conservative diagnostic of what a reviewer would include for remediation, not as the primary evidence that a conflict exists.

Role-level sanity check. After annotation, we compare expert paragraph labels with hidden roles. True endpoints are the only role class with near-universal majority recognition: 95.8% majority clear issue and 100% majority clear-or-relevant. In contrast, affected paragraphs are rarely majority clear issue, and most broader system-core paragraphs are judged ordinary context. This is expected: the benchmark asks methods to retain the compact repair-relevant endpoints, not to mark every retained context paragraph as intrinsically wrong. It also explains why Risk-all is strict: a method may surface useful context or affected nodes while still missing one side of the contradiction.

The example in Table 5 illustrates the intended difficulty level. Neither entity record is a syntactic error by itself. The conflict appears because both records refer to the same consolidation path while asserting mutually incompatible ownership treatments. This is the type of structural conflict that a risk-subgraph method should preserve: a reviewer can repair the issue only after seeing both endpoints.

C Representative Long-SCC Case Study

Figure 5 illustrates the intended use of the method. The original SCC is too large to treat as a single atomic finding, yet too cyclic for ordinary path search. SA-MCGS localizes the evidence through rollouts, then returns a core containing the root, witness, and affected records. This is more useful for review than a single node label: a human can inspect the retained endpoints and decide how the conflict should be repaired.

Audit quantity	Result	Interpretation
Audited critical cases	12 cases, 36 case-level decisions	Each case is independently labeled by three experts.
Majority structural conflict	12/12 (100%)	The audited critical cases are judged to contain structural conflicts.
Repair material sufficiency	10 yes, 1 partial, 1 tie	Most cases contain enough visible material to support repair, but one case remains divided.
No extra context needed	10/12 (83.3%)	Most cases are understandable from the provided paragraphs alone.
Mean confidence	4.28/5	Experts are generally confident in their case-level judgments.
Endpoint majority clear issue	23/24 (95.8%)	Root/witness endpoints are usually judged directly problematic.
Endpoint majority clear-or-relevant	24/24 (100%)	Every endpoint is recognized as at least repair-relevant.
Clear-issue agreement	Fleiss' $\kappa = 0.606$	Agreement is substantial for the binary endpoint-risk judgment.
Repair-packet endpoint-any	7/12 (58.3%)	Majority repair packets include at least one endpoint in over half of cases.
Repair-packet endpoint-all	6/12 (50.0%)	Selecting all endpoints is stricter and less stable across experts.

Table 3: System-blind expert audit results. Experts did not see system outputs, hidden roles, or model-generated hints during annotation.

Task	Expert-facing question	Example expert note
Case-level conflict	Do these paragraphs create a conflict or unresolved high-risk condition when read together?	“The first paragraph gives a right that appears unconditional, but a later paragraph makes the required approval unavailable. The group needs revision before the right can be applied.”
Paragraph label	Does this paragraph look clearly problematic, possibly relevant, ordinary, or uncertain?	“This paragraph is not wrong alone, but it is relevant because it provides the timing condition that controls the later obligation.”
Repair packet	Which paragraphs should a reviewer see to fix the issue?	“The packet should include the grant clause, the later condition clause, and the definition paragraph. Omitting the definition would make the repair ambiguous.”

Table 4: Expert-facing annotation tasks. These are the instructions used to elicit human judgments; internal roles and system outputs are used only for post-hoc analysis.

D Core Algorithm Details

This appendix expands the method into reproducible pseudocode. The implementation uses the same high-level objects as the main paper: a document graph G , strongly connected components S , local rollout windows W_t , relation-first memory M_t , online calibrated conflict signals, and a final dynamic core H_T .

The core is therefore not a monotone union of all suspicious records. Later rollouts can replace weak records with stronger evidence, but the update is not allowed to drop repeatedly supported critical endpoints merely to obtain a smaller sub-graph.

E Formula Details for Evidence Updates

This appendix expands the implementation-level equations that are summarized in Section 3. The main text keeps only the search template, priority score, and core objective; the details below are

Algorithm 2 SCC-localized search

Require: Document graph $G = (V, E)$, rollout budget T , shared risk rubric

- 1: $S \leftarrow$ Tarjan strongly connected components of G
- 2: Initialize collapsed graph $G' \leftarrow G$
- 3: **for all** component $S \in \mathcal{S}$ **do**
- 4: **if** $|S| = 1$ **then**
- 5: Keep the singleton record unchanged
- 6: **else**
- 7: Build local search unit $G_S = (S, E_S)$
- 8: $(H_S, r_S) \leftarrow$ SA-MCGS-ROLLOUT(G_S, T)
- 9: Replace S in G' with collapsed evaluation point (H_S, r_S)
- 10: **end if**
- 11: **end for**
- 12: **return** Collapsed graph G' and all component risk sub-graphs

fixed before evaluation and are not tuned on test cases.

Node and relation updates. For every record observed in rollout t , node risk evidence is updated

Displayed graph	para-	Expert-visible text excerpt	Majority annotation
SEC EX-21 record A	entity	<i>Entergy Louisiana Holdings LLC</i> . “For this consolidation path, the entity is treated as fully consolidated by the upstream registrant. The note states that no separate non-controlling interest is retained for this path.”	Clear issue; included in two experts’ repair packet selections.
SEC EX-21 record B	entity	<i>ENTERGY LOUISIANA HOLDINGS INC</i> . “The same consolidation path reserves a 20 percent non-controlling interest for the downstream schedule. This record applies the opposite state immediately on the same path.”	Clear issue; included in two experts’ repair packet selections.
Case-level decision		The two records describe the same consolidation path but assign incompatible ownership states: fully consolidated with no non-controlling interest versus a retained 20% non-controlling interest.	3/3 experts mark the case as structurally conflicting; 2/3 mark the provided material sufficient, and 1/3 marks it partially sufficient.

Table 5: Representative expert-audit example. Role names in the left column are added here for exposition; the annotators saw paragraph text and identifiers, not root/witness labels or system suggestions.

Algorithm 3 SA-MCGS rollout inside one component

Require: Component $G_S = (S, E_S)$, budget T , shared rubric

- 1: Initialize node evidence, relation evidence, OC evidence, and core $H_0 = \emptyset$
- 2: **for** $t = 1$ to T **do**
- 3: Select a seed record using SCC-aware UCB with relation priors
- 4: Construct local window W_t from the seed, nearby edges, and memory bridges
- 5: Query the LLM with the shared risk rubric on W_t
- 6: Parse local risky records, relation evidence, and optional OC signal
- 7: Update node risk scores, pair evidence, witness-style evidence, and affected-region evidence
- 8: $H_t \leftarrow \text{DYNAMIC-CORE-COMPRESSION}(H_{t-1}, M_t)$
- 9: **end for**
- 10: **return** Final core H_T and ranking induced by accumulated evidence

Algorithm 4 Relation-first evidence memory

Require: Rollout result R_t , previous memory M_{t-1}

- 1: $M_t \leftarrow M_{t-1}$
- 2: **for all** record v observed in R_t **do**
- 3: Update average risk score and visit count for v
- 4: Store why v is risky in relation to visible predecessors or successors
- 5: **end for**
- 6: **for all** reported relation (u, v) in R_t **do**
- 7: Update pair evidence that (u, v) creates or supports a conflict
- 8: Update second-endpoint evidence when v becomes risky because of earlier evidence
- 9: Update affected evidence for downstream records whose status changes
- 10: **end for**
- 11: **if** R_t contains an OC signal **then**
- 12: Add its implicated endpoints and supporting relation to OC memory
- 13: **end if**
- 14: **return** M_t

by a visit-counted running average:

$$\bar{q}_{t+1}(v) = \frac{n_t(v)\bar{q}_t(v) + y_t(v)}{n_t(v) + 1}, \quad (10)$$

$$n_{t+1}(v) = n_t(v) + 1.$$

Directed relation evidence is updated by exponential smoothing,

$$m_{t+1}(u, v) = (1 - \rho)m_t(u, v) + \rho z_t(u, v), \quad (11)$$

where $z_t(u, v)$ is the local evaluator’s evidence that relation (u, v) creates or supports a repair-relevant conflict.

Relation priors and critical pairs. Equation 5 is shared by node and edge priors. For nodes we set $w = \theta$ and use node features such as repair-entry support, local-subgraph appearance, pair-endpoint support, and OC endpoint evidence.

For edges we set $w = \beta$ and use edge features such as semantic incompatibility, repeated pair support, reverse-edge evidence, and accumulated conflict. A separate bounded ledger records persistent critical-pair hypotheses:

$$\mathcal{N}(v) = \{u : (u, v) \in E \text{ or } (v, u) \in E\},$$

$$p_t(v) = \max_{u \in \mathcal{N}(v)} \kappa_t(u, v), \quad (12)$$

$$\kappa_t(u, v) = \min(1, \omega^\top \chi_t(u, v)).$$

Here $\chi_t(u, v)$ contains pair-level evidence such as semantic incompatibility, severity, same-path support, no-fallback language, repeated support, and explicit conflict reports.

Online conflict signal. Let $c_t = \max_{(u,v) \in E(W_t)} z_t(u, v)$ be the strongest relation-conflict score in the current window. With

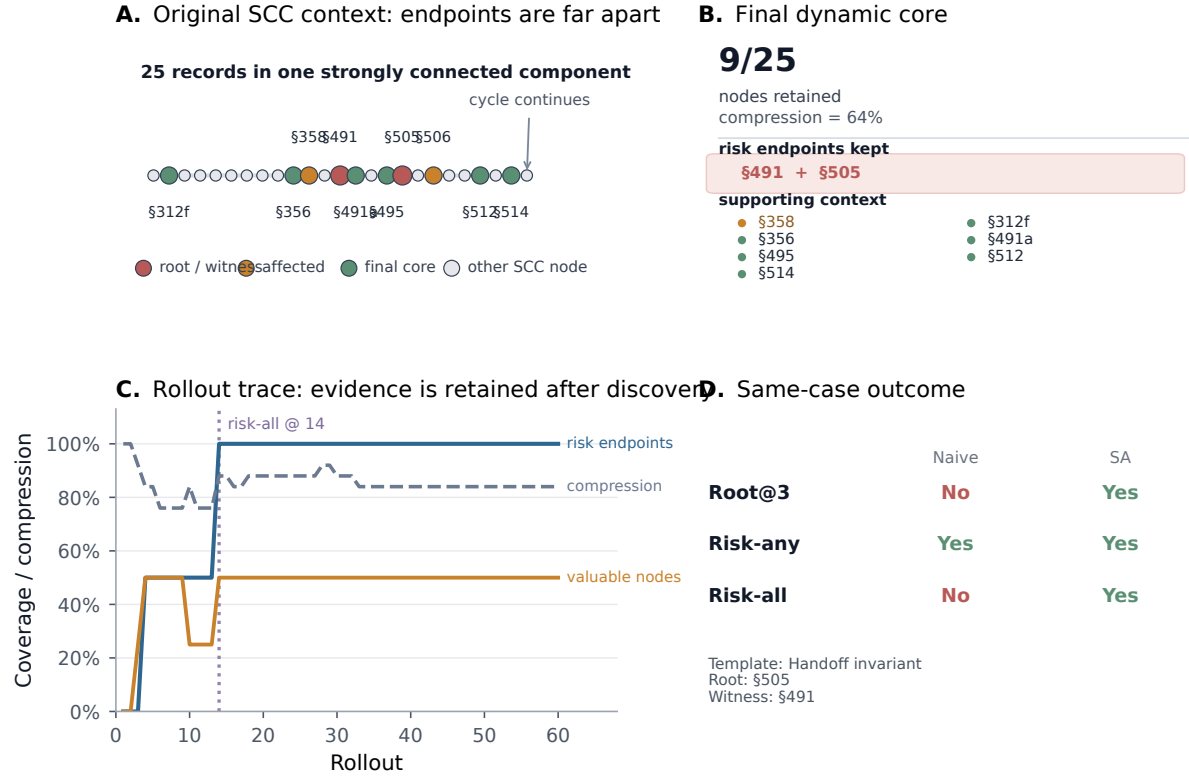


Figure 5: Representative long-SCC collapse. The method searches a large component through local rollouts, accumulates relation-first evidence, and returns a compact risk subgraph that preserves repair-relevant endpoints.

Algorithm 5 Dynamic core compression

Require: Previous core H_{t-1} , memory M_t , candidate cap $B(S)$

- 1: Form candidate set C_t from prior core, local risky records, relation endpoints, affected records, and OC support records
- 2: **for all** $v \in C_t$ **do**
- 3: Score v by node risk, pair support, affected-region support, OC support, and persistence
- 4: Mark v as protected if repeated evidence indicates a critical endpoint
- 5: **end for**
- 6: Start H_t with protected endpoints and strongest relation-supported records
- 7: **while** $|H_t| < B(S)$ and useful candidates remain **do**
- 8: Add the candidate with largest marginal support for explaining the risk
- 9: **end while**
- 10: **while** $|H_t| > B(S)$ **do**
- 11: Remove the weakest non-protected candidate
- 12: **end while**
- 13: **return** H_t

Core candidate pool. The candidate pool used in Eq. 8 is

$$C_t = h_t \cup H_{t-1} \cup O_t \cup K_t, \quad (14)$$

where h_t is the local risk subgraph, O_t contains OC endpoints, and K_t contains top-scoring relation endpoints together with their local neighbors.

F Fixed Search-Prior Coefficients

The numeric values in Table 6 are fixed search-prior coefficients, not learned parameters. They were selected on pilot and smoke runs before the locked evaluation. Domain experts were used only as a sanity check on the ordinal reliability ordering among evidence types, not to tune coefficients on reported test cases. Prior MCTS work motivates the structure: UCB/UCT exploration (Kocsis and Szepesvári, 2006), visit-decayed heuristic priors through progressive bias (Chaslot et al., 2008; Browne et al., 2012), AMAF/RAVE-style evidence reuse (Gelly and Silver, 2007), and graph/transposition-aware value reuse (Childs et al., 2008; Czech et al., 2021). The coefficients instantiate these ideas for structural document risk.

previous rollout scores as the online reference set,

$$\tau_t = Q_{1-\alpha_{oc}}(\{c_i : i < t\}), \quad OC_t = \mathbf{1}[c_t > \tau_t]. \quad (13)$$

The implicated relation endpoints are added to OC memory and receive the bounded bonus in Eq. 7.

Parameter	Value	Role and rationale
<i>Node-feature weights θ for Eq. 5</i>		
θ_{repair}	1.10	LLM names the record as a repair entry; this is closest to the final remediation target.
θ_{local}	0.90	The record appears in local risk subgraphs; useful but noisier than explicit repair-entry evidence.
θ_{pair}	0.85	The record is an endpoint of a supported conflict pair; structural risks often require two endpoints.
θ_{node}	0.70	AMAF/RAVE-style node evidence reused across rollouts; important but indirect.
θ_{path}	0.75	Path-fragment evidence that the record lies on repeatedly implicated local traversals.
θ_{conflict}	0.60	Single-window conflict reports; useful but downweighted because local LLM judgments can be noisy.
θ_{OC}	2.00	Effective ordered-conflict endpoint; high-confidence routing signal, later normalized and visit-decayed.
<i>Edge-feature weights β for Eq. 5</i>		
β_{incompat}	1.55	Semantic incompatibility is the strongest relation-level evidence for structural contradiction.
β_{severity}	1.35	Severity increases priority, but high severity alone is not sufficient without structural support.
β_{explicit}	0.90	Explicit conflict wording helps, but is downweighted relative to persistent structural evidence.
β_{pair}	0.70	Repeated pair support indicates relation-level consistency across windows.
β_{edge}	0.65	AMAF edge evidence reused from related traversals.
β_{reverse}	0.35	Reverse-edge support is weak auxiliary evidence.
β_{conflict}	0.55	The edge’s own accumulated conflict estimate, kept below semantic incompatibility and severity.
<i>Critical-pair prior ω in Eq. 12</i>		
ω_{sem}	0.30	Semantic incompatibility dominates because it directly captures mutual exclusion.
ω_{sev}	0.25	Severity is second: critical risks deserve revisits, but severity alone can be diffuse.
ω_{path}	0.18	Same-path evidence strengthens the claim that two endpoints jointly define one repair region.
$\omega_{\text{nofallback}}$	0.12	No-fallback language makes contradictions harder to resolve locally.
ω_{support}	0.10	Repeated support stabilizes initially uncertain pair hypotheses.
ω_{explicit}	0.05	Explicit conflict reports are useful but lowest-weighted to avoid rewarding one noisy sentence.

Table 6: Fixed search-prior coefficients used by SA-MCGS. Higher values reflect more reliable or more directly repair-relevant evidence. Values were frozen before the locked evaluation and are not learned from the reported test cases.

G Shared Risk Rubric Prompt

Reviewer fairness depends on separating the risk definition from the search mechanism. The following excerpt is the shared evaluator rubric used by both the full-SCC Naive baseline and the SA-MCGS local-window evaluator:

You are given records from a directed graph of interdependent documents. Identify records or record sets that are structurally inconsistent, mutually incompatible, underspecified, or high-impact if left unresolved. Use only visible record text and visible directed relations. Do not assume a domain-specific defect type. Return evidence-based judgments with node ids and concise reasons.

Shared semantics. Naive and SA-MCGS use this same risk rubric. The difference is only the

input unit. Naive reads the complete SCC once and emits a global ranking plus a risk subgraph. SA-MCGS reads local windows over multiple rollouts and aggregates evidence before emitting a final core. Thus the comparison is not between two definitions of risk; it is between one-shot full-context evaluation and localized search with evidence compression.

H Prompt Templates

The complete prompts include serialization details and JSON schemas. We report the abstract templates here because they are sufficient to reproduce the evaluation interface without tying the paper to one implementation file.

Naive full-SCC direct-subgraph prompt.

Input: the complete node list for one strongly connected component, the text of every node, and all directed edges among the nodes. Apply the shared risk rubric to the full component. Return valid JSON with `global_ranking`, `risk_subgraph_nodes`, `risk_factor_distribution`, and `explanation`. Rankings and subgraph selections must be grounded in node IDs and visible edges.

SA-MCGS local-window prompt.

Input: a local window from one strongly connected component, including visible node text, visible directed edges, and a concise prior evidence summary. Apply the same shared risk rubric to the local window. Return valid JSON with `local_node_scores`, `relation_evidence`, `local_risk_subgraph_nodes`, `repair_entry_nodes`, optional `oc_candidate`, and a concise explanation grounded in exact node IDs.

Output contrast. The Naive prompt requires one generation to rank the entire SCC and produce a final subgraph. The SA-MCGS prompt asks for local evidence only; the final subgraph is produced by the algorithmic memory and compression steps rather than by a single LLM response.

I Dataset and Injection Details

Table 7 summarizes why the four domains are included and how they are converted into directed record graphs. The selection is intended to cover different forms of authority and structure: technical dependency metadata, public corporate disclosures, official statutory references, and expert-labeled contract clauses.

SCC construction and filtering. For each domain, we construct a directed record graph and enumerate non-trivial SCCs using Tarjan’s algorithm. Candidate components are filtered by size and record availability, then locked before model evaluation. The main experiment uses the same locked SCC blocks for Naive and SA-MCGS under the same model and profile.

Injection constraints. The critical benchmark modifies only existing records in the selected SCC. It never adds a new node. The root record is the injected endpoint used for Root@3 evaluation. The witness record is a distant evidence endpoint that makes the root-side statement structurally incompatible under the graph context. Affected records are downstream or neighboring records whose interpretation changes because of the root–witness

conflict. Subgraph metrics count root, witness, and affected records as repair-relevant endpoints when applicable.

Why Risk-all matters. Risk-any is useful because a repair process can sometimes start from one good entry point. Risk-all is stricter and is central for critical cases because fixing a structural contradiction often requires both sides of the incompatibility. A subgraph that includes only the visibly alarming paragraph but omits the distant condition may be too small to support a reliable repair.

J Additional Experimental Details

Locked run configuration. Unless otherwise stated, the main tables use the locked profile `current/default + memory_stress(structural_simple_v2) + critical` with 80 SCC blocks per model and four models (GPT-4o, DeepSeek-V3, Qwen2.5-72B, and Gemini 2.5 Pro). The same SCC block is evaluated by the full-SCC direct-subgraph Naive baseline and by SA-MCGS; strict rates count unusable structured outputs in the denominator.

Domain	Why selected	Graph construction	Role in evaluation
Debian	Technical dependency graph with explicit package relationships.	Package metadata entries become records; dependency, replacement, conflict, and virtual-provider fields become directed edges.	Tests whether the method generalizes beyond legal and financial language.
SEC EX-21	Public company subsidiary disclosures with entity-control structure.	Subsidiary names, jurisdictions, consolidation fields, ownership hints, and control relations become entity records and directed edges.	Tests public regulatory disclosure reasoning and ownership-chain consistency.
BGB	Official German Civil Code cross-references.	Statutory sections become records; references, obligations, conditions, exceptions, and remedies become directed edges.	Provides a comparatively clean legal-rule stress test with authoritative source text.
CUAD	Expert-labeled commercial contract dataset.	Clauses and definitions become records; definition use, cross-reference, amendment, condition, termination, and obligation relations become edges.	Provides long, noisy contract components where evidence can be diffuse.

Table 7: Dataset construction and processing. All domains are represented as directed record graphs before SCC localization.

Template	Root-side edit	Witness / affected edit	Intended risk
direct_mutex	Adds a direct obligation, ownership, compatibility, or condition statement that must hold.	Adds a distant statement that requires the opposite state under the same dependency chain.	Two records are individually plausible but mutually incompatible when the component context is followed.
handoff_invariant	States that an invariant must be preserved before control, migration, or responsibility is handed off.	States that the downstream record can proceed only after the invariant has been changed or removed.	The handoff cannot be completed because the precondition and downstream entry condition disagree.
temporal_gate	Specifies a deadline, waiting period, migration window, vesting condition, or trigger order.	Requires a conflicting temporal gate, such as earlier execution, delayed activation, or a missing prerequisite.	The same chain imposes incompatible timing constraints.
condition_trigger	Makes a right, remedy, interface, or ownership statement conditional on a trigger.	Makes the same trigger impossible, optional, or reversed for a dependent record.	A dependent record relies on a condition whose truth value is changed elsewhere in the component.

Table 8: Critical structural injection templates. All templates edit existing records only and avoid explicit defect labels.

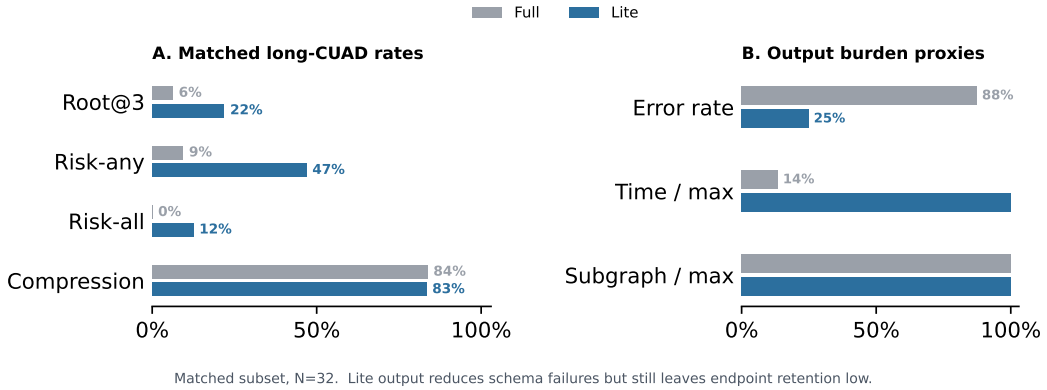


Figure 6: Naive output-burden control. Reducing the output schema helps full-SCC prompting, but does not close the endpoint-retention gap on long CUAD components.

Output reliability	Naive	SA-MCGS
Method records	320	320
Usable structured outputs	261/320	320/320
Unavailable structured outputs	59/320	0/320
Usable-output rate	82%	100%

Table 9: Strict output reliability. Unavailable structured outputs remain in the denominator of Figure 3; the table is placed in the appendix because output availability is an audit detail rather than a headline metric.

A. Representative critical cases

Domain	Component	Template	Conflict summary	Repair material should include
BGB	25-record statute	condition_trigger or temporal_gate	One section states when an obligation or remedy applies; a distant referenced section changes the trigger or timing in a way that makes the remedy inconsistent.	The remedy section, distant condition, and any bridge section carrying the condition.
CUAD	18–25-record contract	direct_mutex or handoff_invariant	A clause preserves a right, notice obligation, or approval condition; a later clause makes the prerequisite unavailable or contradicts the pre-served condition.	The granting clause, conflicting condition clause, and any definition that determines applicability.
SEC EX-21	9–18-record entity	direct_mutex	One disclosure states full consolidation or no minority interest; another record in the same control chain retains a non-controlling interest or incompatible ownership state.	The consolidation record, ownership witness, and affected subsidiaries when present.
Debian	package dependency	handoff_invariant	A package keeps an old ABI or compatibility surface during migration, while a dependent package requires the new interface to exist first.	The invariant package, dependent package, and handoff relation that makes ordering impossible.

B. Output availability

Scope	Method	N	Root@3	Risk-all
Strict	Naive	320	48%	47%
Strict	SA-MCGS	320	81%	78%
Valid-only	Naive	261	59%	57%
Matched	Naive	261	59%	57%
Matched	SA-MCGS	261	83%	77%

C. Model-level strict breakdown

Model	Method	N	Err.	Root@3	Risk-any	Risk-all	Comp.
GPT-4o	Naive	80	17	26%	55%	15%	68%
GPT-4o	SA-MCGS	80	0	86%	100%	75%	51%
DeepSeek-V3	Naive	80	0	75%	88%	74%	70%
DeepSeek-V3	SA-MCGS	80	0	80%	95%	76%	51%
Qwen2.5-72B	Naive	80	41	2%	46%	14%	43%
Qwen2.5-72B	SA-MCGS	80	0	70%	98%	75%	53%
Gemini 2.5 Pro	Naive	80	1	90%	98%	85%	75%
Gemini 2.5 Pro	SA-MCGS	80	0	88%	98%	84%	56%

D. Domain-level strict breakdown

Domain	Method	N	Err.	Root@3	Risk-any	Risk-all	Comp.
Debian	Naive	32	0	41%	94%	62%	56%
Debian	SA-MCGS	32	0	84%	100%	88%	53%
SEC EX-21	Naive	80	0	64%	99%	62%	59%
SEC EX-21	SA-MCGS	80	0	96%	100%	88%	50%
BGB	Naive	48	2	25%	67%	25%	62%
BGB	SA-MCGS	48	0	75%	98%	62%	52%
CUAD	Naive	160	57	49%	55%	42%	79%
CUAD	SA-MCGS	160	0	74%	96%	75%	54%

Table 10: Supplementary case and result breakdowns. The representative cases illustrate why a single suspicious record is often insufficient for remediation. Strict rates count unusable structured outputs in the denominator; matched usable-output rates show that the difference is not only output availability.